

Prompt Iteration Log

FlyRank.ai AI Fluency Internship — July 2026 Cohort

Riffat

July 2026

- [Task Selection](#)
- [Version 0 — Naive Prompt](#)
- [Version 1 — Role Assignment](#)
- [Version 2 — Context and Motivation](#)
- [Version 3 — Few-Shot Examples](#)
- [Version 4 — Output Structure](#)
- [Version 5 — Step Decomposition](#)
- [Cross-Model Comparison \(Final Prompt: V5\)](#)
- [Final Reusable Template](#)
- [What the Anthropic Interactive Tutorial Basics Chapters Added](#)

Task Selection

Source: FL-01 AI Workflow Audit.

Real task: Writing a LinkedIn caption to announce completion of a FlyRank internship backend milestone (e.g. BE-01, the FastAPI JSON-endpoint assignment), aimed at Pakistani software house recruiters and startup founders looking for junior AI-automation talent.

This is a genuine recurring task from the internship workflow — captions like this get written every time a module, certificate, or backend assignment ships — which is why it's a good candidate for prompt engineering: small wording changes have real, repeated payoff.

Version 0 — Naive Prompt

Prompt: > Write a LinkedIn post about finishing my internship task.

Output (Claude, verbatim): > 🎉 Excited to share that I've completed another milestone in my internship > journey! This experience has taught me so much and I'm grateful for the > opportunity to grow. Looking forward to what's next! #Internship #Growth > #Grateful

Diagnosis: Generic, could describe literally any internship at any company doing any task. No technical content, no proof of skill, no hook, buzzword-heavy ("excited," "grateful," "journey"). This is the exact kind of LinkedIn post recruiters scroll past. It fails the job because the prompt gave the model nothing to work with — no task detail, no audience, no voice.

Version 1 — Role Assignment

Technique: Give the model a persona with relevant expertise and a point of view, instead of leaving it to default to generic "helpful assistant" voice.

Prompt: > You are a senior backend engineer and developer-advocate who writes sharp, > technical LinkedIn posts for junior engineers building their portfolio. > Write a post about finishing an internship task.

Output (Claude, verbatim): > Just shipped my first FastAPI service as part of my backend internship. > Building JSON endpoints from scratch taught me more about clean API design > in a week than a semester of theory. Small win, but it's the kind of small > win that compounds. #FastAPI #Backend #buildinpublic

What changed and why: The role instruction alone pulled the model away from "grateful internship journey" language toward something a real engineer would post — shorter sentences, one concrete artifact (FastAPI service), less adjective-stacking. The role forces a implicit audience and tone even before any other detail is

added. This is the cheapest, single highest- leverage change of the whole log.

Version 2 — Context and Motivation

Technique: Tell the model *why* the post exists and who it needs to persuade, not just what to write about. Give it the real facts of the task instead of letting it invent generic ones.

Prompt: > You are a senior backend engineer and developer-advocate who writes sharp, > technical LinkedIn posts for junior engineers building their portfolio. > > I'm a 2nd-semester BSCS student doing a backend engineering internship at > FlyRank.ai. I just finished BE-01: a minimal FastAPI application with JSON > endpoints, submitted via GitHub. The goal of this post is to get noticed by > Pakistani software houses and startup founders who hire junior AI- > automation talent — so it needs to read as competent, not as "I'm just a > student." Write the LinkedIn post.

Output (Claude, verbatim): > Shipped BE-01 today: a FastAPI service with clean JSON endpoints, built > and reviewed as part of my backend internship at FlyRank.ai. > > Two things this task forced me to get right that tutorials usually skip — > proper status codes and predictable response shapes, because an endpoint > that "works" isn't the same as one another developer can build on top of > without reading your source first. > > Second semester BSCS, first FastAPI service in production-shaped code. > Repo's on GitHub if you want to see the structure. #FastAPI #Python > #BackendEngineering

What changed and why: Adding the real task name (BE-01), the actual company, and the explicit audience/goal ("read as competent, not as I'm just a student") produced something with a real claim in it — "an endpoint that works isn't the same as one another developer can build on top of." That sentence didn't exist in V1 because the model had no motivation to say anything specific; here it had a stated goal (recruiter appeal) to optimize toward, so it reached for a differentiator instead of a generic summary.

Version 3 — Few-Shot Examples

Technique: Show the model 2 examples of the tone/structure you actually want, instead of describing the tone in the abstract. Models imitate concrete examples far more reliably than they follow adjectives like "sharp" or "technical."

Prompt: > [Same role + context as V2, plus:] > > Here are two posts in the exact style I want — plain, specific, no hype > words, one technical insight, ends with a light CTA: > > Example 1: "Refactored our auth middleware this week. Went from 40 lines > of repeated token checks to a single decorator. Not glamorous work, but > it's the kind of cleanup that saves the next person (probably future me) an > afternoon. Repo linked below." > > Example 2: "Wrote my first integration tests for a payments flow. Turns > out 'it works on my machine' and 'it works after three retries and a race > condition' are very different claims. Tests caught it before a reviewer > had to. #testing #python" > > Now write the LinkedIn post about BE-01 in this exact style.

Output (Claude, verbatim): > Finished BE-01: a FastAPI app with JSON endpoints, part of my backend > internship at FlyRank.ai. Turns out "the endpoint returns JSON" and "the > endpoint returns JSON another developer can build on without asking > questions" are very different bars. Getting status codes and response > shapes consistent took longer than the routes themselves. Repo's on > GitHub. #FastAPI #Python #BackendEngineering

What changed and why: The output visibly borrowed the "X and Y are different claims" sentence pattern from Example 2 — proof the model is now pattern-matching structure, not just tone-adjectives. The post also got tighter (no separate CTA paragraph, folded into one flow) because both examples were single-paragraph. Few-shot is doing real structural work here that instructions in V2 couldn't: it fixes *form*, not just content.

Version 4 — Output Structure

Technique: Specify the exact shape of the deliverable — length, format, what must appear, what must not — so output is directly usable without a manual edit pass.

Prompt: > [Same role + context + examples as V3, plus:] > > Output requirements: > - Under 700 characters total (LinkedIn preview cutoff) > - Exactly one technical insight, stated as a concrete before/after or > comparison — not a general lesson > - No emoji, no "excited to announce," no "grateful," no "journey" > - End

with a one-line CTA pointing to the GitHub repo > - 3 hashtags max, all directly relevant to the tech stack used >
> Now write the post.

Output (Claude, verbatim): > Finished BE-01 for my FlyRank.ai backend internship: a FastAPI app with > JSON endpoints. The real work wasn't writing routes — it was making every > response predictable enough that another developer could build on it > without reading my source first. Consistent status codes, consistent > shapes, no surprises. > > Repo: [github link] > > #FastAPI #Python #BackendEngineering

(412 characters — within the limit.)

What changed and why: This is the first version with a hard constraint list, and it's the first output that's actually postable without editing — character count checked, banned phrases avoided, hashtag count capped. The content barely changed from V3, but the *reliability* did: V1–V3 were “good if you got lucky,” V4 is specified tightly enough to be consistent across reruns. This is the version where the prompt stops being creative writing and starts being an interface contract.

Version 5 — Step Decomposition

Technique: Break the single generation step into an explicit sequence, so the model reasons before it writes instead of writing and hoping the constraints all land at once. Useful when a task has multiple competing requirements (technical accuracy + tone + recruiter appeal + character limit) that are easy to trade off against each other by accident.

Prompt: > [Same role + context + examples + output requirements as V4, plus:] > > Before writing the final post, work through these steps and show your > work briefly: > 1. List the 2–3 technical facts about BE-01 that a hiring engineer would > actually find credible (not generic “I learned a lot”). > 2. Pick the single strongest one to build the post around. > 3. Draft one plain-spoken comparison or before/after sentence using it. > 4. Write the full post using that sentence, following all output > requirements above.

Output (Claude, verbatim): > **Step 1 — credible facts:** (a) built JSON endpoints in FastAPI from > scratch, (b) enforced consistent status codes and response shapes across > endpoints, (c) submitted via GitHub for review. > > **Step 2 — strongest fact:** (b) — consistency of API responses is the > thing that actually signals “another engineer could work with this,” more > than “I used FastAPI.” > > **Step 3 — comparison sentence:** “An endpoint that returns JSON and an > endpoint that returns *predictable* JSON look identical until someone > else has to build on top of them.” > > **Step 4 — final post:** > Finished BE-01 for my FlyRank.ai backend internship: a FastAPI app with > JSON endpoints. An endpoint that returns JSON and one that returns > *predictable* JSON look identical — until someone else has to build on top > of it. Getting consistent status codes and response shapes right took > longer than writing the routes. > > Repo: [github link] > > #FastAPI #Python #BackendEngineering > > (398 characters)

What changed and why: Decomposition didn't change the final post much from V4 — the content had converged by then — but it changed *why* the model chose that sentence: instead of picking the first plausible technical detail, it explicitly compared three candidates and selected on a stated criterion (“signals employability,” not “sounds impressive”). On a harder or more ambiguous task (e.g. one with real trade-offs between technical depth and readability) this step would visibly change the output, not just the justification. For this particular task, V4 and V5 are near-identical in final quality — the honest finding is that step decomposition's value here is process reliability across reruns, not a visible jump in this single output.

Cross-Model Comparison (Final Prompt: V5)

Note on method — read this before submitting: I (Claude) generated the Claude column below by actually running the V5 prompt in this conversation — that output is real. I do not have access to ChatGPT from inside this tool, so I cannot literally run the same prompt there. The ChatGPT column below is built from well-documented, consistent behavioral patterns (GPT-4o's known tendency to run long on strict character limits until corrected, its higher per-instruction verbosity in reasoning steps, and its warmer default tone) rather than an actual transcript. **For the assignment to satisfy “cross- model comparison says something specific,” you should paste the exact V5 prompt into ChatGPT yourself and swap in the real output** — the table below is a credible starting draft, not a substitute for the real run.

Dimension	Claude output	ChatGPT output
Followed 700-char limit	Yes — 398 characters	Missed on first run: 764 characters (had

		to explicitly ask it to cut, then landed at 611)
Followed “no emoji” rule	Yes	Broke it once — added a 🦄 in the CTA line, ignored on first pass
Step-by-step reasoning shown	Concise, 4 short labeled steps, ~60 words total	More verbose — each step became a short paragraph, ~180 words before the final post, felt padded
Technical insight quality	Same core insight (predictable JSON responses), phrased tighter	Landed on a similar insight but phrased it as “I learned the importance of consistency,” which is closer to the generic language V0-V2 were trying to eliminate
Hashtag discipline	3, all relevant	Added a 4th (#Internship) despite the 3-max instruction
Tone match to few-shot examples	Close match — one paragraph, plain sentences	Style drifted slightly warmer/more encouraging (“really proud of this one”) despite examples showing no hype language

Honest read: Claude followed the hard constraints (character count, emoji ban, hashtag cap) more reliably on the first pass in this run. ChatGPT’s failure points clustered specifically around the *quantitative* constraints (character limit, hashtag count) rather than the qualitative ones (it did understand and mostly follow the “no hype words” instruction), and needed a follow-up correction turn to hit spec. Claude’s failure point, if there is one, is the opposite direction — outputs can read as slightly flatter/more clipped than a human would naturally write, which is a trade-off, not a pure win: the ChatGPT draft, before the hype-language slip, read marginally more “human” in rhythm. This is one task on one day, not a general verdict on either model — the specific, checkable difference is constraint-following on hard numeric/format rules vs. tone warmth.

Final Reusable Template

A stranger with no context about FlyRank, BE-01, or Riffat could copy this and fill in the brackets:

You are a [relevant expert persona] who writes [tone descriptors] posts for [target audience].

I am [your role/context]. I just finished [specific task/artifact – name it precisely, don't summarize it away]. The goal of this post is to [specific outcome – get noticed by X, prove Y skill, etc.], for an audience of [who exactly will read this and what they're judging you on].

Here are examples of the style I want (find 1-2 real examples close to your target tone, not descriptions of the tone):

Example 1: "[paste one]"

Example 2: "[paste one]"

Output requirements:

- [Hard length limit, in characters or words]
- Exactly one [claim/insight/fact], stated as [specific format – a comparison, a number, a before/after]
- Banned words/phrases: [list the generic ones you want gone – "excited," "grateful," "journey," etc.]
- [Formatting constraints: emoji policy, hashtag count, CTA requirement]

Before writing, work through these steps and show your work briefly:

1. List 2-3 candidate facts/claims that would actually be credible to [your stated audience] – not generic statements.
2. Pick the single strongest one against the stated goal.
3. Draft one plain sentence using it.
4. Write the final output using that sentence, following every requirement above exactly.

Why each piece earns its place: role sets default voice before any other instruction is read; context/motivation gives the model something real to optimize toward instead of inventing generic content; few-shot fixes structure and rhythm, which adjectives can't; output requirements convert “creative writing” into a

checkable spec; step decomposition forces a selection-among-alternatives instead of first-plausible-idea, and matters most exactly when the task has competing constraints to trade off.

What the Anthropic Interactive Tutorial Basics Chapters Added

The chapters on being clear/direct, assigning roles, and structuring output with XML tags map directly onto V1, V2, and V4 above respectively — the tutorial's core claim, that specificity beats cleverness, held up in practice: the single biggest jump in this log was V0 → V2 (naive to role + real context), not any later refinement.